# Section 6: Two Pointers

This section is part of the 5-hour LeetCode DSA Foundations course by Mukul Raina

- YouTube Course: https://youtu.be/YJMBv4Q2Tg8?si=Q1vDzusUJf753GG3
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/


C11: LeetCode DSA Foundations for Interviews

Section 6: Two Pointers

- **SLIDE 51: Section Intro, Two Pointers**
- **SLIDE 52: The Two-Pointer Mental Model: From O(n^2) to O(n)**
- **SLIDE 53: Opposite-Direction Pointers**
- **SLIDE 54: Same-Direction and Multi-Pointer Patterns**
- **SLIDE 55: When to Use Two Pointers vs Hash Map**
- **SLIDE 56: Worked Example: Three Sum**
- **SLIDE 56B: Three Sum: Trace and Analysis**
- **SLIDE 57: Wrong vs Right: Two Pointer Mistakes**
- **SLIDE 58: How to Present Two Pointer Solutions in an Interview**
- **SLIDE 59: Interview Practice: Two Pointers**
- **SLIDE 60: Section Cheat Sheet**


# C11: LeetCode DSA Foundations for Interviews

# Section 6: Two Pointers

## SLIDE 51: Section Intro, Two Pointers

### Section 6: Two Pointers

Two pointers is a technique where two index variables traverse a data structure in a coordinated way, eliminating the need for nested loops. Instead of checking every possible pair with O(n^2) brute force, two pointers exploits structure in the input (usually sorted order) to find the answer in O(n).

The core idea: if the input has some ordering property, you do not need to check every combination. You can make an informed decision about which pointer to move, discarding large portions of the search space with each step.

**What We'll Cover:**

1. The two-pointer mental model and why it converts O(n^2) to O(n)
2. Opposite-direction pointers (left/right moving inward)
3. Same-direction pointers (read/write for in-place operations)
4. Decision framework: two pointers vs hash map
5. Worked example: Three Sum with duplicate handling
6. How to present two-pointer solutions in interviews

**Where This Fits in the Course:**

Two pointers extends the array techniques from Section 3 and often serves as an alternative to the hash map approach from Section 5. The difference: hash maps trade O(n) space for O(1) lookups on unsorted input, while two pointers achieves O(n) time with O(1) space on sorted input. Sliding window (Section 7) is a specialized form of same-direction two pointers applied to subarray optimization problems. Binary search (Section 8) also operates on sorted input but solves a different class of problems (finding a specific element or boundary rather than finding pairs).

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

## SLIDE 52: The Two-Pointer Mental Model: From O(n^2) to O(n)

### Why Brute Force is Slow

When searching for a pair of elements that satisfies some condition (e.g., two numbers that sum to a target), the brute force checks every pair using nested loops. For an array of n elements, there are roughly n^2/2 pairs. Each one is checked independently, with no information carried from one check to the next.

### Why Two Pointers is Fast

On sorted input, each comparison gives you directional information. If the sum of the two pointed-to elements is too small, you know that moving the left pointer rightward increases the sum. If the sum is too large, moving the right pointer leftward decreases it. Each step eliminates an entire row or column of possible pairs, not just one.

Each pointer moves at most n positions total. Combined movement: at most 2n steps. Total: O(n).

### Side-by-Side: Brute Force vs Two Pointers

Brute force O(n^2), works on unsorted input:

```java
public int[] twoSumBrute(int[] nums, int target) {
    for (int i = 0; i < nums.length; i++) {
        for (int j = i + 1; j < nums.length; j++) {
            if (nums[i] + nums[j] == target) {
                return new int[]{i, j};
            }
        }
    }
    return new int[]{};
}
```

Two pointers O(n), requires sorted input:

```java
public int[] twoSumSorted(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left < right) {
```

```
            int sum = nums[left] + nums[right];
            if (sum == target) {
                return new int[]{left, right};
            } else if (sum < target) {
                left++;
            } else {
                right--;
            }
        }
        return new int[]{};
    }
```

**Approach Comparison:**

| Approach | Time | Space | Prerequisite | Returns |
|---|---|---|---|---|
| Nested loops | O(n^2) | O(1) | None | Original indices |
| Hash map (Section 5) | O(n) | O(n) | None | Original indices |
| Two pointers | O(n) | O(1) | Sorted input | Positions in sorted array |

Two pointers gives you the best of both worlds (O(n) time AND O(1) space) but demands sorted input. If you need original indices from unsorted input, use the hash map. If the problem gives you sorted input or only asks for values, two pointers is the strongest choice.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

## SLIDE 53: Opposite-Direction Pointers

### The Pattern

One pointer starts at the beginning, one at the end. They move toward each other based on a condition. The loop ends when they meet or cross.

**Template:**

```
int left = 0, right = arr.length - 1;
while (left < right) {
    // Evaluate condition using arr[left] and arr[right]
    if (conditionMet) {
        // Record result or return
    } else if (needLargerValue) {
        left++;
    } else {
        right--;
    }
}
```

The decision of which pointer to move depends on the problem. The sorted order guarantees that moving left increases the left value and moving right decreases the right value.

**When to Recognize This Variant:**

- Sorted array + find a pair satisfying a condition
- Symmetric comparison from both ends (palindromes)
- Optimization over a range that narrows (container problems)

**Classic Problems:**

| Problem | Condition | Move Left When | Move Right When |
|---|---|---|---|
| Two Sum II (sorted) | sum == target | sum < target | sum > target |
| Valid Palindrome | chars match from both ends | characters match (both move) | characters match (both move) |
| Container With Most Water | maximize area | left height < right height | right height <= left height |
| Three Sum (inner loop) | two-pointer inside outer loop | sum < 0 | sum > 0 |
| Trapping Rain Water | accumulate water | left max < right max | right max <= left max |

**Worked Inline: Container With Most Water (LeetCode 11)**

Given n vertical lines where line i has height `height[i]`, find two lines that together with the x-axis form a container holding the most water.

The area between lines at positions left and right is `min(height[left], height[right]) * (right - left)`. Start with the widest container (left = 0, right = n-1). The only way to potentially find a larger area is to increase the height, which means moving the shorter side inward.

```
public int maxArea(int[] height) {
    int left = 0, right = height.length - 1;
    int maxWater = 0;
    while (left < right) {
        int width = right - left;
        int h = Math.min(height[left], height[right]);
        maxWater = Math.max(maxWater, h * width);
        if (height[left] < height[right]) {
            left++;
        } else {
            right--;
        }
    }
    return maxWater;
}
```

**Why moving the shorter side is correct:** Moving the taller side can only decrease or maintain the height (min is still capped by the shorter side) while definitely decreasing the width. That can never increase the area. Moving the shorter side might find a taller line, which could increase the area despite the narrower width.

Time O(n), space O(1).

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

## SLIDE 54: Same-Direction and Multi-Pointer Patterns

### Same-Direction Pointers (Read/Write)

Both pointers start at the beginning. A fast pointer reads every element. A slow pointer marks the write position where valid elements go. Elements before the slow pointer are the processed result. Elements at or after the slow pointer are unprocessed or overwritten.

**Template:**

```
int write = 0;
for (int read = 0; read < arr.length; read++) {
    if (isValid(arr[read])) {
        arr[write] = arr[read];
        write++;
    }
}
// arr[0..write-1] contains valid elements
```

**When to Recognize:**

- "In place" or "O(1) extra space" in the problem statement
- "Remove", "deduplicate", "move", "partition" operations
- The problem modifies an array without allocating a new one

**Worked Inline: Move Zeroes (LeetCode 283)**

Move all zeroes to the end of the array while maintaining the relative order of non-zero elements. Do it in place.

```
public void moveZeroes(int[] nums) {
    int write = 0;
    for (int read = 0; read < nums.length; read++) {
        if (nums[read] != 0) {
            int temp = nums[write];
            nums[write] = nums[read];
            nums[read] = temp;
            write++;
        }
```

```
        }
    }
```

Trace through [0, 1, 0, 3, 12]:

```
Initial:        [0, 1, 0, 3, 12]   write=0

read=0: nums[0]=0, skip
read=1: nums[1]=1, swap(0,1) → [1, 0, 0, 3, 12]   write=1
read=2: nums[2]=0, skip
read=3: nums[3]=3, swap(1,3) → [1, 3, 0, 0, 12]   write=2
read=4: nums[4]=12, swap(2,4) → [1, 3, 12, 0, 0]  write=3

Result: [1, 3, 12, 0, 0]
```

Time O(n), space O(1). This pattern reuses the read/write pointer technique introduced in Section 3 (Slide 22).

**Fast-Slow Pointers (Brief Introduction)**

One pointer moves at speed 1, another at speed 2. When used on linked lists, fast-slow pointers detect cycles and find the middle element. This variant is covered in full in Section 11 (Linked Lists).

Two key applications:

- Cycle detection (Floyd's algorithm): if a cycle exists, the fast pointer eventually catches the slow pointer inside the cycle
- Find middle: when the fast pointer reaches the end, the slow pointer is at the midpoint

These applications require a linked list structure, not arrays. If you see "detect cycle" or "find middle of linked list" in a problem, fast-slow pointers is the technique.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

## SLIDE 55: When to Use Two Pointers vs Hash Map

### Decision Framework

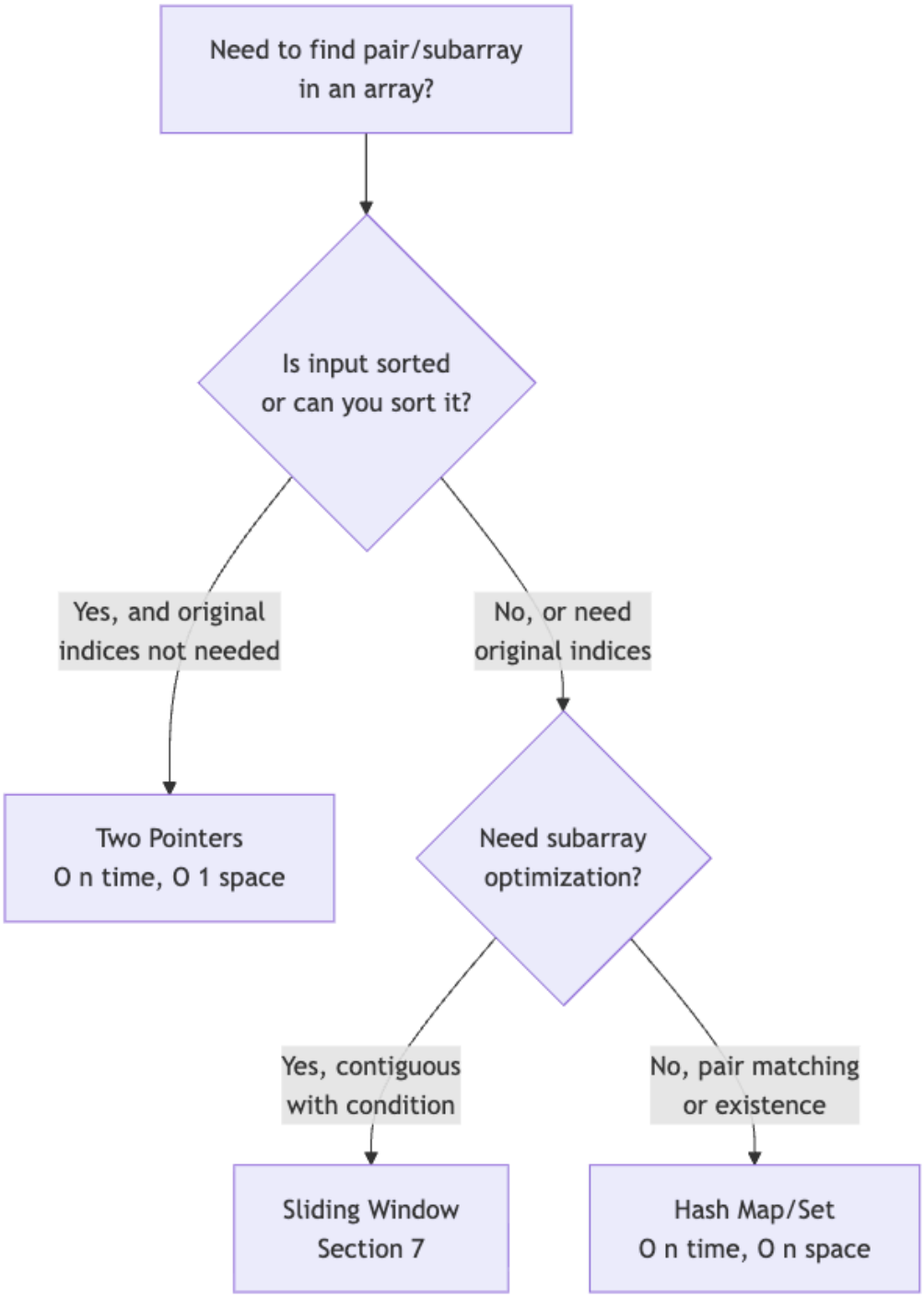
The same problem can often be solved by either two pointers or a hash map. The right choice depends on the input characteristics and the problem constraints.

| Input Characteristic | Problem Type | Best Approach | Why |
|---|---|---|---|
| Sorted array | Find pair with target sum | Two pointers | O(1) space, exploits sorted order |
| Unsorted array, need indices | Find pair with target sum | Hash map | Cannot sort without losing original indices |
| Unsorted array, need boolean | Any value appears twice? | HashSet | Simpler than sorting + two pointers |
| Sorted array, need all pairs | Find all pairs summing to target | Two pointers | O(1) space, handle duplicates by skipping |
| Unsorted, optimize subarray | Longest/shortest subarray with condition | Sliding window (Section 7) | Specialized two-pointer variant |
| String, symmetric check | Palindrome verification | Two pointers | Natural fit, O(1) space |
| In-place modification | Remove/move/deduplicate | Same-direction pointers | O(1) space is the constraint |

**When Two Pointers Does NOT Apply:**

- Unsorted input where sorting would lose information you need (original indices)
- Problems that need all possible pairs, not just one or a count (output size is O(n^2), no technique avoids generating all of them)
- Subarray optimization with a dynamic constraint like “sum <= target” where the window can expand and shrink. That is sliding window (Section 7), not basic two pointers.
- Problems where the relationship between elements is not monotonic. Two pointers works because moving a pointer in one direction predictably changes

the condition (larger or smaller sum). If the effect of moving a pointer is unpredictable, two pointers cannot eliminate possibilities.


Need to find pair/subarray in an array?
Is input sorted or can you sort it?
Yes, and original indices not needed
No, or need original indices
Two Pointers O n time, O 1 space
Need subarray optimization?
Yes, contiguous with condition
No, pair matching or existence
Sliding Window Section 7
Hash Map/Set O n time, O n space

**Interview Tip:** The phrase “sorted array” in a problem statement is almost always a signal for two pointers or binary search. When you see it, mention both options to the interviewer: “Since the array is sorted, I can use two pointers for O(1) space, or binary search if I am looking for a single element.” This shows you understand the design space.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

## SLIDE 56: Worked Example: Three Sum

**Problem: Three Sum (LeetCode 15)**

Given an integer array `nums`, return all unique triplets [nums[i], nums[j], nums[k]] such that i != j != k and nums[i] + nums[j] + nums[k] == 0.

The result must not contain duplicate triplets.

**Step 1: Understand**

Input: unsorted integer array, may contain duplicates. Output: list of unique triplets summing to zero. “Unique” means no duplicate triplets in the result, even if the same values appear multiple times in the input. Example: [-1, 0, 1, 2, -1, -4] should return [[-1, -1, 2], [-1, 0, 1]], not [[-1, 0, 1], [-1, 0, 1]].

**Step 2: Identify the Pattern**

Brute force: three nested loops checking all triplets. O(n^3). Can we do better?

Fix one element with an outer loop. The remaining problem is: “find two elements in the rest of the array that sum to negative of the fixed element.” That is Two Sum. On sorted input, Two Sum is O(n) with two pointers. So: sort first, outer loop fixes one element, inner two pointers find the pair.

Total: O(n log n) sort + O(n) outer loop x O(n) inner two pointers = O(n^2). This is optimal for Three Sum (proven lower bound).

**Step 3: Plan**

1. Sort the array

2. For each element nums[i] as the fixed element:
    - Skip if nums[i] == nums[i-1] (avoid duplicate triplets from same first element)
    - Set target = -nums[i]
    - Two pointers from i+1 to end to find pairs summing to target
    - When a pair is found, skip duplicate values for both pointers
3. Return collected triplets

**Step 4: Implement**

```
public List<List<Integer>> threeSum(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    Arrays.sort(nums);

    for (int i = 0; i < nums.length - 2; i++) {
        if (nums[i] > 0) break;
        if (i > 0 && nums[i] == nums[i - 1]) continue;

        int left = i + 1, right = nums.length - 1;
        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (sum == 0) {
                result.add(Arrays.asList(nums[i], nums[left],
nums[right]));
                while (left < right && nums[left] == nums[lef
t + 1]) left++;
                while (left < right && nums[right] == nums[ri
ght - 1]) right--;
                left++;
                right--;
            } else if (sum < 0) {
                left++;
            } else {
                right--;
```

```
                }
            }
        }
        return result;
    }
```

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

## SLIDE 56B: Three Sum: Trace and Analysis

### Step 5: Verify

Trace through [-1, 0, 1, 2, -1, -4]:

```
After sorting: [-4, -1, -1, 0, 1, 2]

i=0: nums[0]=-4, target=4
  left=1(-1), right=5(2): sum = -4 + -1 + 2 = -3 < 0 → left++
  left=2(-1), right=5(2): sum = -4 + -1 + 2 = -3 < 0 → left++
  left=3(0),  right=5(2): sum = -4 + 0 + 2 = -2 < 0  → left++
  left=4(1),  right=5(2): sum = -4 + 1 + 2 = -1 < 0  → left++
  left=5, left >= right → stop

i=1: nums[1]=-1, target=1
  left=2(-1), right=5(2): sum = -1 + -1 + 2 = 0 → FOUND [-1,-1,2]
    skip duplicate lefts (none), skip duplicate rights (none)
    left=3, right=4
  left=3(0), right=4(1): sum = -1 + 0 + 1 = 0  → FOUND [-1,0,1]
    left=4, right=3, left >= right → stop

i=2: nums[2]=-1, same as nums[1] → skip (duplicate first element)

i=3: nums[3]=0 > 0? No. target=0
  left=4(1), right=5(2): sum = 0 + 1 + 2 = 3 > 0 → right--
  left=4, right=4, left >= right → stop

Result: [[-1,-1,2], [-1,0,1]]
```

### Complexity:

- Time: O(n^2). Sorting is O(n log n). The outer loop runs n times, inner two-pointer scan is O(n) per iteration. O(n log n) + O(n^2) = O(n^2).
- Space: O(1) auxiliary (excluding the output list and the space used by the sort). Java's Arrays.sort on primitives uses O(log n) stack space.

**Why Duplicate Handling Works:**

Three duplicate checks prevent the same triplet from appearing twice:

| Check | What It Prevents | Example |
|---|---|---|
| `if (i > 0 && nums[i] == nums[i-1]) continue` | Same first element producing same triplets | [-1, -1, 0, 1]: both -1s would find [0, 1] |
| `while (nums[left] == nums[left+1]) left++` | Same second element in a found triplet | [-2, 1, 1, 1]: after finding [-2, 1, 1], skip remaining 1s on left |
| `while (nums[right] == nums[right-1]) right--` | Same third element in a found triplet | [-2, 0, 1, 1, 2, 2]: after finding pair, skip duplicate 2s on right |

**Early Termination Optimization:**

The line `if (nums[i] > 0) break` is an optimization. After sorting, if the current fixed element is positive, no two elements to its right (also positive or zero) can sum to a negative target. All remaining triplets would sum to a value greater than zero. This is a small optimization but demonstrates awareness in an interview.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

## SLIDE 57: Wrong vs Right: Two Pointer Mistakes

### Mistake 1: Applying Two Pointers to Unsorted Input

Wrong approach:

```
// Array is NOT sorted, but candidate uses two pointers anywa
y
int left = 0, right = nums.length - 1;
while (left < right) {
```

```
        int sum = nums[left] + nums[right];
        if (sum == target) return new int[]{left, right};
        else if (sum < target) left++;
        else right--;
    }
```

This fails on unsorted input like [3, 1, 4, 2] with target 5. The pointers might skip the pair (1, 4) entirely because the movement logic assumes sorted order.

Right: Sort first if using two pointers, or use a hash map (Section 5) if you need original indices from unsorted input.

**Mistake 2: Not Explaining WHY the Pointer Moves**

Wrong: "I move left pointer right."

Right: "The current sum is smaller than the target. Because the array is sorted, moving the left pointer rightward increases the left value, which increases the sum toward the target. Moving the right pointer leftward would decrease the sum further, so that is the wrong direction."

The interviewer needs the reasoning, not just the action. Two pointers is a technique where the correctness depends entirely on the movement justification.

**Mistake 3: Forgetting to Include Sorting Cost**

Wrong: "Two pointers is O(n) time."

Right: "I sort the array first in O(n log n), then the two-pointer scan is O(n). The total time complexity is O(n log n). The sort dominates."

If the input is already given as sorted (like Two Sum II on LeetCode), then the solution is O(n). But if you sort it yourself, the sort cost is part of your solution.

**Mistake 4: Using Two Pointers When Indices Are Needed**

The original Two Sum (LeetCode 1) asks for indices in the original unsorted array. Sorting rearranges elements, losing their original positions.

Wrong: Sorting and using two pointers, then returning left and right as indices. These are positions in the sorted array, not the original.

Right: Either use a hash map (O(n) time, O(n) space, preserves indices), or create an indexed copy that tracks original positions (as shown in Section 2, Slide 17,

Solution 2). The hash map approach is simpler and preferred when indices are needed.

**Mistake 5: Not Handling Duplicates in Three Sum**

Wrong: Skipping the duplicate checks, producing results like [[-1,0,1], [-1,0,1]].

Right: Three skip checks as detailed in Slide 56B. When the interviewer says “no duplicate triplets”, the duplicate-skipping logic is part of the solution, not an afterthought.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

## SLIDE 58: How to Present Two Pointer Solutions in an Interview

**Introducing the Approach**

When the problem gives sorted input or when you decide to sort:

“Since the array is sorted, I can use two pointers starting at opposite ends. The sorted order gives me a guarantee: if the current sum is too small, I know moving the left pointer right will increase it, and if too large, moving the right pointer left will decrease it. This lets me find the pair in O(n) instead of O(n^2).”

**For Three Sum:**

“I will fix one element with an outer loop. The remaining problem reduces to Two Sum on sorted input, which I can solve with two pointers. I will sort first to enable this. The total is O(n^2): O(n) for the outer loop times O(n) for the inner two-pointer scan, plus O(n log n) for the sort which is dominated by O(n^2).”

If the interviewer asks about duplicates: “I handle duplicates at three points: skip duplicate fixed elements in the outer loop, and skip duplicate left and right values after finding a valid triplet.”

**Comparing to Hash Map**

When both approaches are viable, briefly mention both and explain your choice:

“I could use a hash map here for O(n) time and O(n) space on unsorted input. But since I can sort the array and the problem does not require original indices, two pointers gives me O(n) time with O(1) auxiliary space. The sort costs O(n log n) but that is acceptable.”

This shows the interviewer you considered alternatives and made a deliberate trade-off decision.

**Explaining the Same-Direction Variant**

“The problem asks me to modify the array in place with O(1) extra space. I will use a read pointer and a write pointer, both starting at the beginning. The read pointer scans every element. When it finds a valid element, it copies it to the write position and advances write. Everything before write is the processed result.”

**When to Draw a Diagram**

Two-pointer problems benefit from a visual trace, especially Three Sum and Container With Most Water. On a whiteboard, draw the array, mark the pointer positions, and show the movement at each step. This takes 30 seconds and prevents mistakes during implementation.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

## SLIDE 59: Interview Practice: Two Pointers

**Practice Prompt**

“Given a sorted array of integers, find two numbers that add up to a specific target. Return their indices (1-indexed).” (LeetCode 167: Two Sum II)

**Expected Walkthrough:**

Step 1 (Understand): “The input is already sorted and there is exactly one solution. I need to return 1-indexed positions.”

Step 2 (Identify): “Sorted input plus pair finding maps directly to opposite-direction two pointers. I could also use a hash map, but two pointers takes advantage of the sorted order for O(1) space.”

Step 3 (Plan): “Left pointer at index 0, right pointer at the last index. Compare their sum to target. If equal, return. If too small, move left right. If too large, move right left.”

Step 4 (Implement):

```
public int[] twoSum(int[] numbers, int target) {
    int left = 0, right = numbers.length - 1;
    while (left < right) {
        int sum = numbers[left] + numbers[right];
        if (sum == target) {
            return new int[]{left + 1, right + 1};
        } else if (sum < target) {
            left++;
        } else {
            right--;
        }
    }
    return new int[]{};
}
```

Step 5 (Verify): Trace through [2, 7, 11, 15] with target 9.

```
left=0(2), right=3(15): sum=17 > 9 → right--
left=0(2), right=2(11): sum=13 > 9 → right--
left=0(2), right=1(7):  sum=9 == 9 → return [1, 2]
```

“Time complexity O(n), space complexity O(1). Each pointer moves at most n positions, and we use no auxiliary data structures.”

**Follow-Up: “What if the array is not sorted?”**

“If the array is unsorted and I need original indices, sorting would lose the index mapping. I would use a HashMap storing value-to-index, checking for the complement at each step. That gives O(n) time with O(n) space. Two pointers is preferable when the input is already sorted because it avoids the space cost.”

**Common Fresher Mistakes:**

- Not leveraging the “sorted” constraint. If you reach for a hash map on sorted input, you are leaving the O(1) space advantage on the table. The interviewer may follow up with “can you do it without extra space?” and the answer is the two-pointer approach you should have started with.
- Applying two pointers to problems where the constraint is not monotonic. Two pointers works because the sorted order creates a predictable relationship between pointer movement and the sum. If moving a pointer could increase or decrease the condition unpredictably, two pointers is the wrong tool.
- Off-by-one on 1-indexed vs 0-indexed return values. Read the problem statement carefully. Two Sum II is 1-indexed, original Two Sum is 0-indexed.
- Forgetting to mention the O(1) space advantage. The whole reason to use two pointers over a hash map on sorted input is the space saving. State it explicitly.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

## SLIDE 60: Section Cheat Sheet

### Two Pointer Variant Decision Table

| Problem Signal | Variant | Pointer Start Positions | Movement Logic |
|---|---|---|---|
| Sorted array + find pair with condition | Opposite-direction | left = 0, right = n-1 | Move based on comparison to target |
| Palindrome check | Opposite-direction | left = 0, right = n-1 | Both move inward when chars match |
| Maximize area/container | Opposite-direction | left = 0, right = n-1 | Move the shorter/smaller side |
| In-place remove/move/deduplicate | Same-direction (read/write) | both at 0 | Write advances only on valid elements |

| Problem Signal | Variant | Pointer Start Positions | Movement Logic |
|---|---|---|---|
| Cycle detection in linked list | Fast-slow | both at head | Slow +1, fast +2 |
| Find middle of linked list | Fast-slow | both at head | Slow +1, fast +2 |
| Three Sum | Outer loop + opposite-direction | i fixed, left = i+1, right = n-1 | Standard two-pointer inside outer loop |

### Template Code Quick Reference

Opposite-direction (pair finding on sorted input):

```
int left = 0, right = arr.length - 1;
while (left < right) {
    int val = evaluate(arr[left], arr[right]);
    if (val == target) return result;
    else if (val < target) left++;
    else right--;
}
```

Same-direction (in-place filtering):

```
int write = 0;
for (int read = 0; read < arr.length; read++) {
    if (isValid(arr[read])) {
        arr[write++] = arr[read];
    }
}
```

### Key Complexity Facts

| Variant | Time | Space | Prerequisite |
|---|---|---|---|
| Opposite-direction | O(n) | O(1) | Sorted input (or sort first for O(n log n) total) |
| Same-direction | O(n) | O(1) | None, works on unsorted |
| Fast-slow | O(n) | O(1) | Linked list structure |

| Variant | Time | Space | Prerequisite |
|---|---|---|---|
| Three Sum (with sort) | O(n^2) | O(1) auxiliary | Sorted input (sort first) |

### Pattern Recognition Signals

| Signal in Problem Statement | First Pattern to Try |
|---|---|
| “Sorted array” + “find pair” | Opposite-direction two pointers |
| “In place” + “remove” or “move” | Same-direction read/write pointers |
| “Cycle in linked list” | Fast-slow pointers (Section 11) |
| “Middle of linked list” | Fast-slow pointers (Section 11) |
| “Three numbers that sum to…” | Sort + outer loop + inner two pointers |
| “Container” or “trapping water” | Opposite-direction, move shorter side |
| “Palindrome” | Opposite-direction, compare from ends |
| “Sorted array” + “find element” | Binary search (Section 8), not two pointers |
| “Unsorted” + “need original indices” | Hash map (Section 5), not two pointers |

### How This Connects to Other Sections

- Section 3 (Arrays): the read/write pointer technique for in-place operations was introduced there and formalized here as same-direction two pointers
- Section 5 (Hash Maps): hash maps and two pointers solve overlapping problem sets. Hash maps work on unsorted input with O(n) space. Two pointers work on sorted input with O(1) space. Know both and choose based on constraints.
- Section 7 (Sliding Window): sliding window is a specialized same-direction two-pointer technique for contiguous subarray optimization. If the problem says “longest/shortest subarray with condition”, that is Section 7, not this section.
- Section 8 (Binary Search): both operate on sorted input. Two pointers finds pairs. Binary search finds individual elements or boundaries. If the problem is “find one element” in sorted input, use binary search. If the problem is “find a pair”, use two pointers.

- Section 11 (Linked Lists): fast-slow pointers are covered in full there. This section introduced the concept; Section 11 provides the implementations.

**Common Interview Mistakes**

| Mistake | Why It's Wrong | Better Approach |
|---|---|---|
| Two pointers on unsorted input | Movement logic assumes sorted order | Sort first, or use hash map |
| Not stating why pointer moves in that direction | Interviewer cannot evaluate your reasoning | "Sum is too small, moving left increases it because array is sorted" |
| Saying "O(n)" when solution sorts first | Forgot the O(n log n) sort cost | "O(n log n) total: O(n log n) sort + O(n) scan" |
| Two pointers when original indices are needed | Sorting loses index mapping | Hash map preserves indices on unsorted input |
| No duplicate handling in Three Sum | Produces duplicate triplets in result | Three skip checks: outer loop, left after match, right after match |

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina